

CS7DS2 Assignment - Week 2

Name: Kehan Liu ID:25334065

Q1

i. A

Differentiating a function using SymPy $f(x)=x^4$ is no different from regular mathematical differentiation. First, I define the symbolic variable as x , and then use the `diff` function to differentiate the expression.

```
x = sp.symbols('x')
f = x ** 4
f_prime = sp.diff(f, x)
```

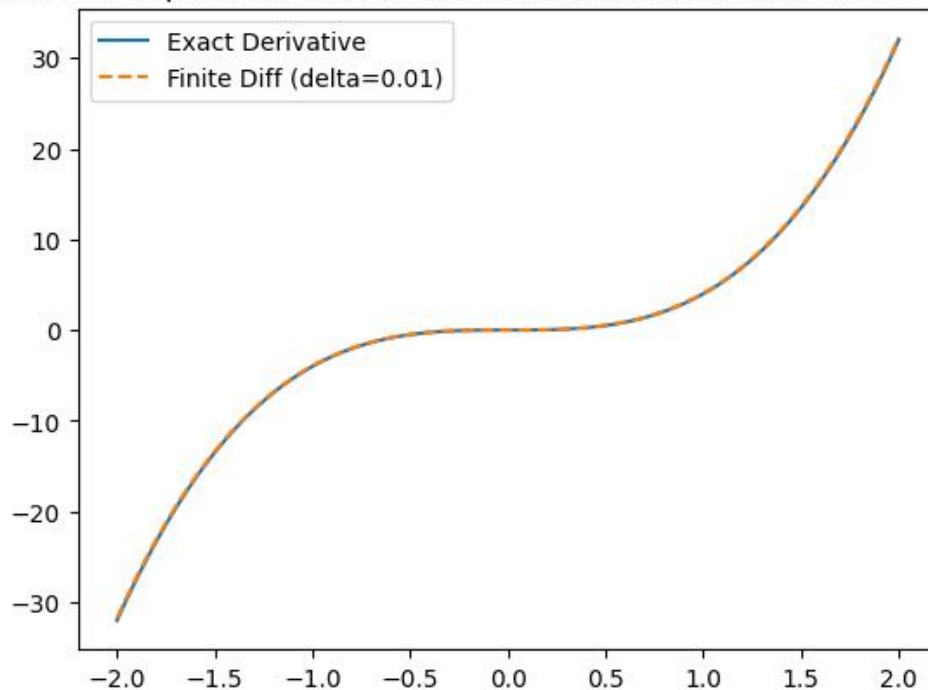
The final result is $f'(x)=4x^3$ consistent with the rules for differentiating power functions in mathematics, so no further explanation is needed.

i. B

Unlike i.a, the finite difference approximation actually takes two extremely close points and calculates the slope between them. The mathematical formula is $f'(x) \approx \frac{f(x+\delta) - f(x)}{\delta}$, where δ is the x -coordinate distance between the two points δ . The smaller the slope, the closer it is to the accurate result.

The results of i.a and i.b are visualized as shown in the figure below.

Q1.1B Comparison of Exact Derivative and Finite Difference ($\delta = 0.01$)



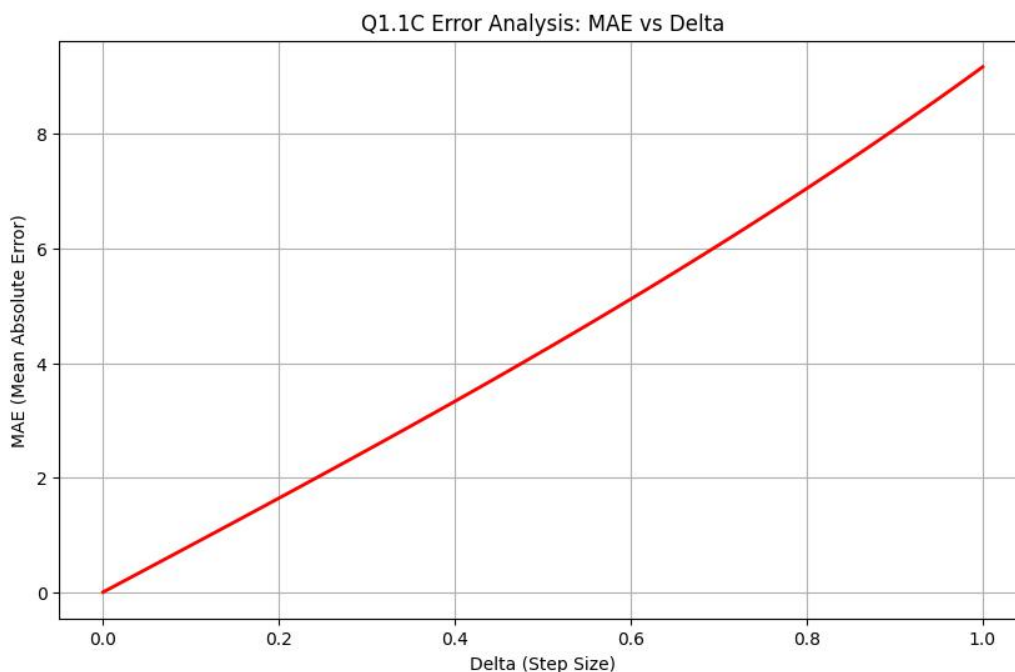
As can be seen, because the value is δ small enough, the two lines almost overlap. This proves that the finite difference method approximates the true derivative value very accurately in this case. Of course, this is also related to the range of X values $[-2,2]$. Perhaps when X is large enough, we can see this subtle difference.

I. C

As I mentioned in ib, the accuracy of the finite difference method is related to the value of δ . To verify this relationship, I took 100 values in $[0.001, 1]$, cyclically substituted these 100 values into the finite difference formula in ib, calculated the difference between it and the exact derivative, and calculated the MSE.

```
deltas = np.linspace( 0.001 , 1 , 100 )
mae_values = []
for d in deltas :
    approx = forward_difference ( x_vals , d )
    error = np.abs ( y_exact ) - approx )
    mae = np.mean ( error )
    mae_values.append ( mae )
```

The visualization results are shown in the figure below.



increases linearly with increasing step size δ . I believe this is because the finite difference method essentially approximates the slope of the tangent line with the slope of the secant line. As the step size increases, the distance between the two ends x and $x + \delta$ of the secant line becomes greater, resulting in a larger truncation error.

I. D

Gradient descent is an iterative optimization algorithm with the following update rule :

$x_{k+1} = x_k - \alpha f'(x_k)$, where x_k is the position at the k -th iteration.

α The learning rate controls the step size. $f'(x_k)$ is the gradient of the function at k . Substituting this

into the function in this problem $x_{k+1} = x_k - \alpha \cdot (4x_k^3)$:

In the code implementation of gradient descent, there is a loop that iteratively updates the variable x . In each iteration, the algorithm calculates the gradient at the current position and then updates the

position using the rule $x_{k+1} = x_k - \alpha \cdot (4x_k^3)$. I used the same initial value $x_0 = 1$ and tested three different learning rates $\{0.05, 0.5, 1.2\}$, iterating 30 times.

```
def gradient_descent ( x0 , alpha , iterations ):
    history_x = [ x0 ]
    history_f = [ f ( x0 )]
```

```

x_current = x0
for i in range ( iterations ):
    grad = df_exact ( x_current )
    x_next = x_current - alpha * grad
    history_x.append ( x_next )
    history_f.append ( f ( x_next ))
    x_current = x_next
return history_x , history_f

```

When the learning rate is 0.05, the final position is around 0.239. Compared to the initial position of 1, there has been convergence. I think this is because the step size is small enough to ensure

$f(x_{k+1}) < f(x_k)$ the update amount in each iteration $\Delta x = -\alpha \nabla f(x_k)$.

When the learning rate is 0.5, the final position remains at 1, unchanged. From a mathematical perspective, at $x_0=1$, the gradient... $\nabla f(1)=4(1)^3=4$

Update formula: $x_1=1-0.5 \times 4=-1$, at $x_1=-1$, gradient $\nabla f(-1)=4(-1)^3=-4$

was updated: $x_2=-1-0.5 \times (-4)=1$ This resulted in an infinite loop.

It's worth noting that when the learning rate is 1.2, the program throws an overflow error. This is because at $x_0=1$, the update position is: $x_1 = -3.8$.

This means that the new position is farther from the bottom than the old position . Since the original function is a power function, when the number of iterations is too many, it will explode exponentially and overflow.

```

Learning rate 0.05: Final position x = 0.2686951602124061
Learning rate 0.5: Final position x = 1.0

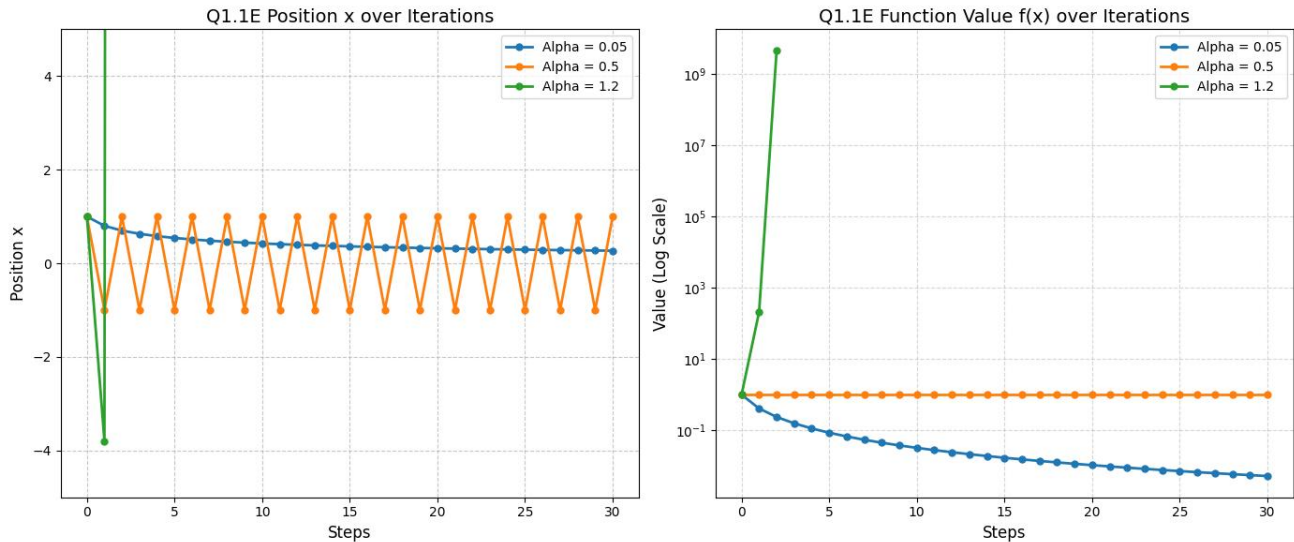
-----
OverflowError                                Traceback (most recent call last)
Cell In[6], line 19
     16 results = {}
     18 for a in alphas:
--> 19     x_hist, f_hist = gradient_descent(x0=1, alpha=a, iterations=30)
     20     results[a] = (x_hist, f_hist)
     22     print(f"Learning rate {a}: Final position x = {x_hist[-1]}")

```

This result shows that the gradient descent algorithm requires a suitable learning rate; too large a rate will lead to slow learning speed, and too large a rate will lead to oscillations or even divergence .

i. E

The results of i.D are visualized as shown in the figure below.



The graph on the right confirms our mathematical inference about iD: when the learning rate is 0.05, x gradually converges; when the learning rate is 0.5, the position of x fluctuates between -1 and 1; and when the learning rate is 1.2, overflow occurs.

Focusing on the graph on the right, when the learning rate is 0.05, $f(x)$ gradually decreases, which indicates convergence and the error is steadily decreasing .

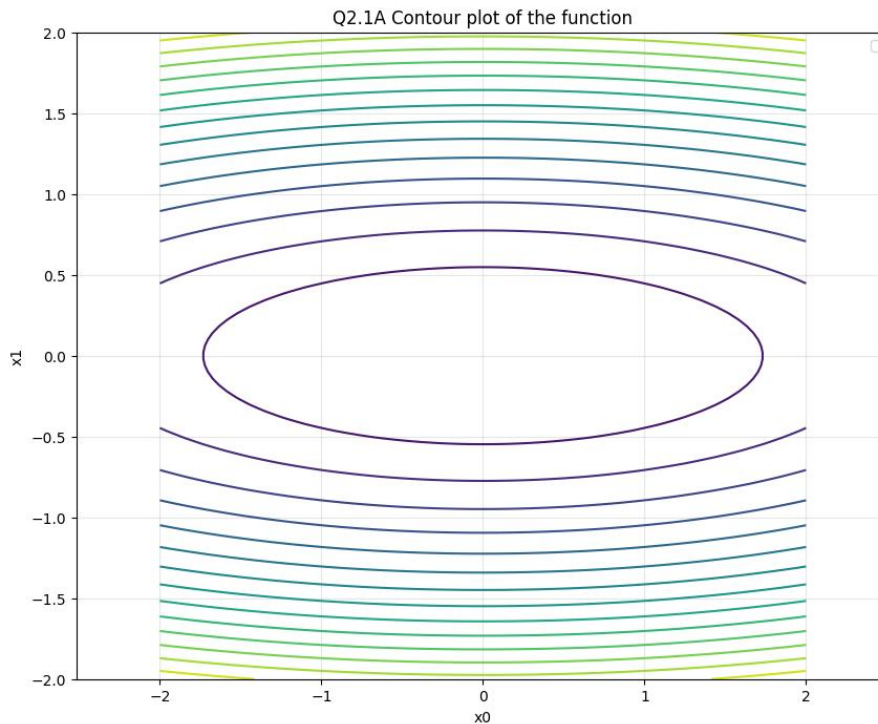
When the learning rate is 0.5, $f(x)$ is a straight line because X repeats between -1 and 1 during the iteration, while the original function is a fourth power. Therefore, $f(x)$ does not change at all, which indicates that the algorithm has stalled and has not produced any effect.

When the learning rate is 1.2, $f(x)$ increases sharply, indicating divergence . With each step, the error rate actually increases , suggesting that the learning rate is too large and has a counterproductive effect on performance improvement.

Q2

I. A

Contour analysis of the function $f(x) = \frac{1}{2}(x_0^2 + 10x_1^2)$ shows that the original function is a valley resembling an ellipse. The contour lines along the x_1 axis are very dense, indicating a steep slope; while the contour lines along the x_0 axis are relatively sparse, indicating a gentler slope.



I. B

Take the partial derivative with respect to X_0 : $\frac{\partial f}{\partial x_0} = \frac{1}{2} \cdot (2x_0) = x_0$

Taking the partial derivative with respect to X_1 : $\frac{\partial f}{\partial x_1} = \frac{1}{2} \cdot (20x_1) = 10x_1$, therefore the derivative of the original function is $\nabla f = \begin{bmatrix} x_0 \\ 10x_1 \end{bmatrix}$

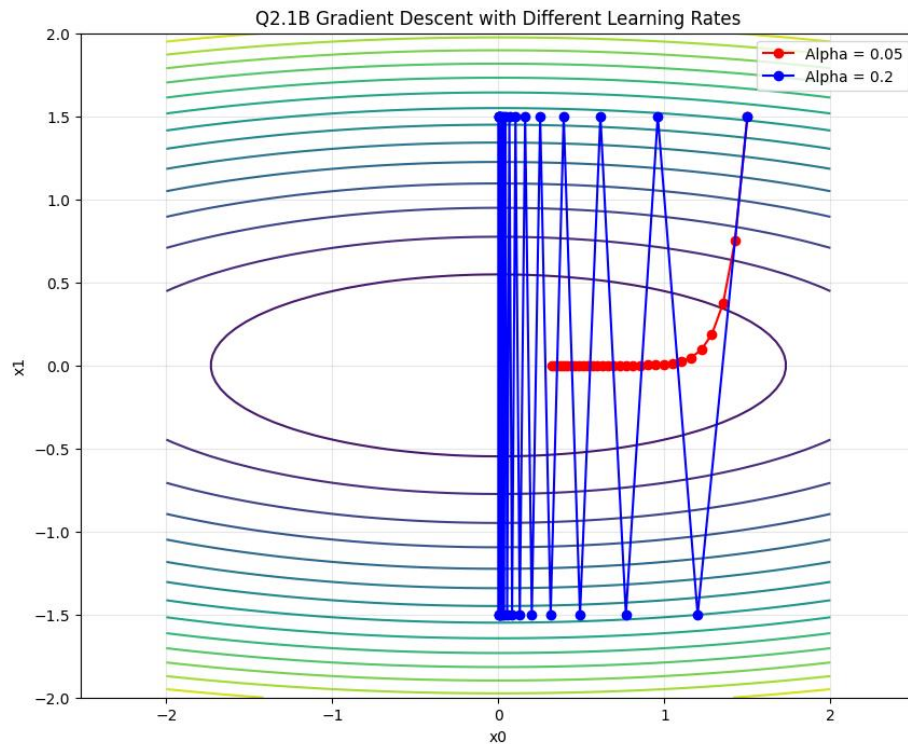
The code first calculates the reciprocal of the original position, then $x_{k+1} = x_k - \alpha \nabla f(x_k)$ calculates the new position based on the gradient descent update formula, and iterates multiple times. Unlike in Q1, here I replace the two parameters with a NumPy vector using the array function to facilitate subsequent calculations.

```
def grad_2d ( x ):
    return np.array ( [ x [ 0 ], 10... ] * x [ 1 ] )
def gd_2d ( start_point , alpha , iterations ):
    path = [ start_point ]
    x_curr = np.array ( start_point )
    for _ in range ( iterations ):
        g = grad_2d ( x_curr )
        x_next = x_curr - alpha * g
        path.append ( x_next )
        x_curr = x_next
    return np.array ( path )
```

The final results show that the red path (learning rate 0.05) flows smoothly towards the center of the circle without any fluctuations and converges stably. This is because the learning rate is small and can adapt to the steep X_1 direction.

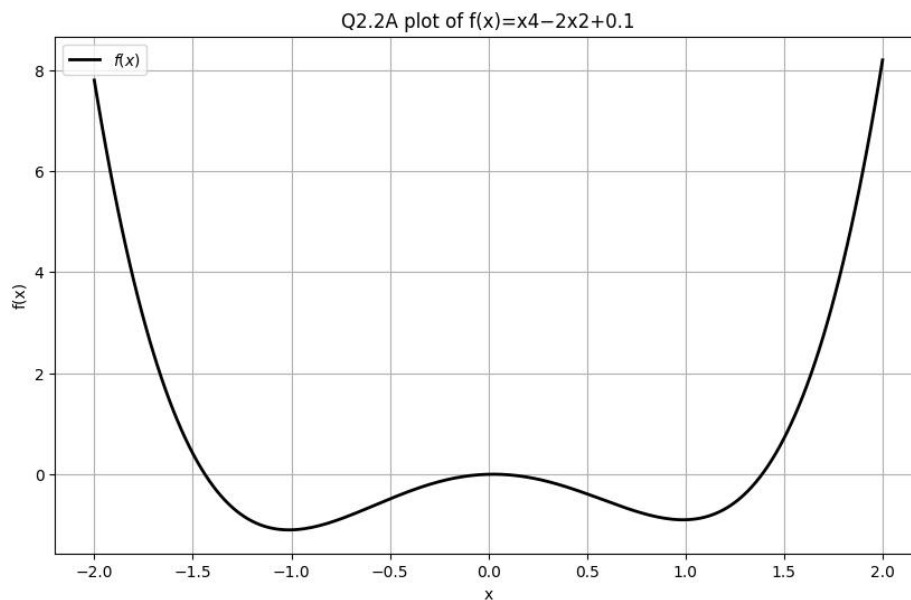
The blue path (learning rate 0.2) exhibits jagged edges and jumps wildly along the X_1 axis . I believe this is because the gradient is very large in the X_1 direction. The step size of 0.2 results in excessive update size $\Delta x_1 = -0.2 \times (10x_1) = -2x_1$, meaning that each update jumps to the opposite position and changes the sign , hindering effective descent.

Clearly, although a smaller learning rate converges more slowly at X_0 , it is more stable and performs much better than a larger learning rate.



Ii. A

The visualization of the function is as follows; $f(x)=x^4-2x^2+0.1$ its shape resembles the letter "W", with a trough on each side.



Ii.B

Differentiate the original function: $f(x)=4x^3-4x+0.1$, the code logic here is exactly the same as Q1, first differentiate, then use the gradient descent update function, and iterate multiple times.

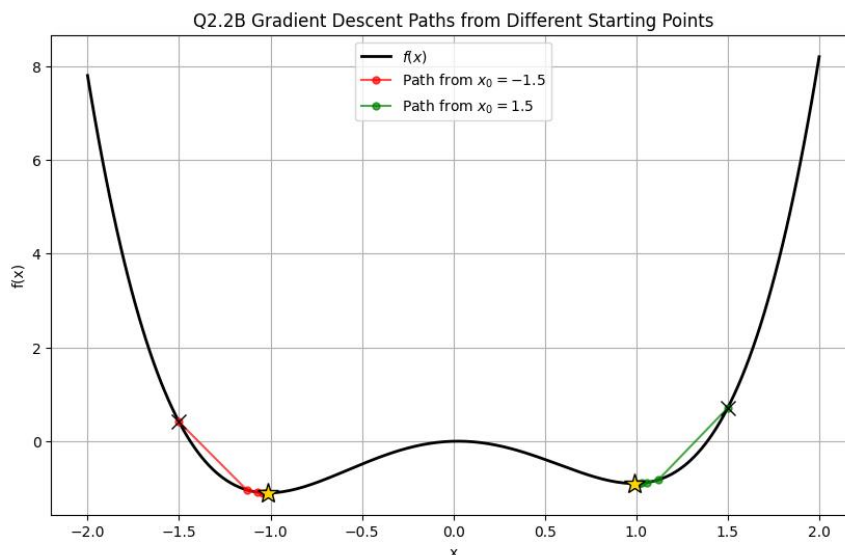
```
def gradient_descent_1d ( x_start , alpha , num_iters = 30 ):
    path = [ x_start ]
    x_curr = x_start
    for _ in range ( num_iters ):
        grad = grad_f_1d ( x_curr )
```

```

x_next = x_curr - alpha * grad
path.append ( x_next )
x_curr = x_next
return np.array ( path )

```

is at two different positions, $x_0 = -1.5$ and $x_0 = 1.5$. The final result is shown in the figure below. It can be seen that the red path (starting point -1.5) finally arrives at the deep valley on the left, which is the actual lowest point. However, the blue path (starting point 1.5) falls into a misconception and stops at the bottom of the valley on the right. This is a local minimum, and the derivative is also 0. The function mistakenly thinks it has reached the lowest point and stops. This illustrates that the quality of the solution is highly dependent on the choice of initial values. The gradient descent algorithm can only guarantee finding a local minimum, but not the global minimum.



Q3

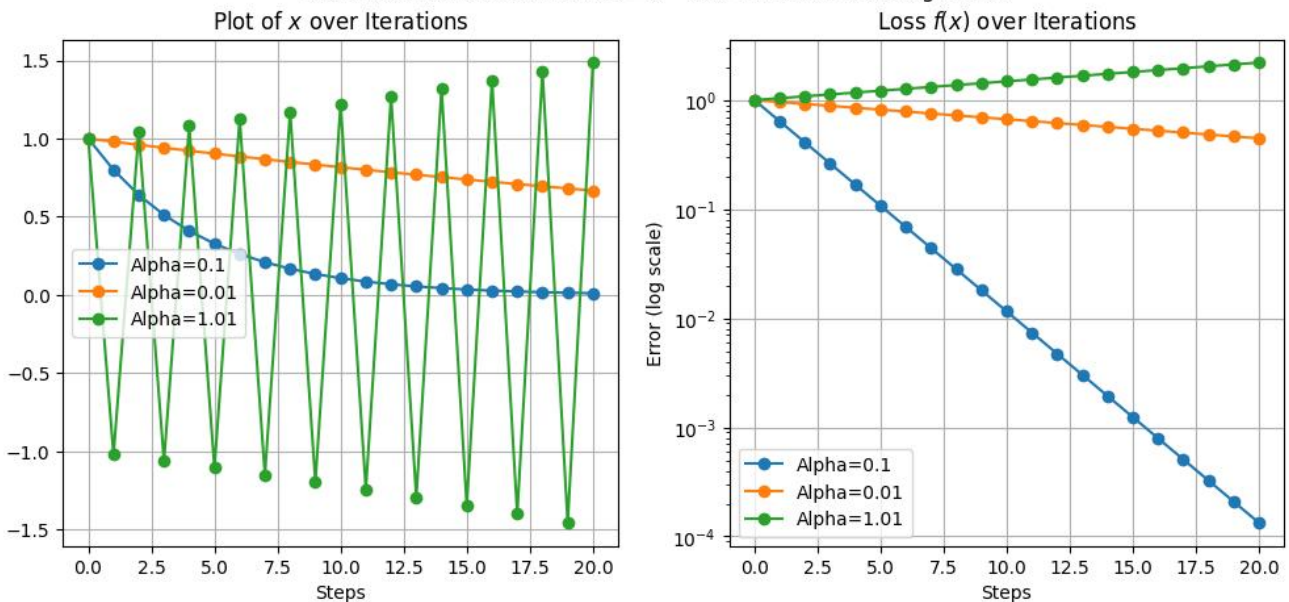
I. AB

The original function $f(x)=x^2$, its reciprocal $f'(x)=2x$, and the corresponding gradient descent function $x_{k+1}=x_k(1-2\alpha)$.

with different learning rates $\alpha = 0.1$, 0.01 , and 1.01 , and the results are shown in the figure below. The left figure shows the result when $\alpha = 0.1$. The value of X decreases smoothly and monotonically from 1 to 0. Analyzing the gradient descent function we just obtained, each descent decreases by 20%. When $\alpha = 0.01$, the value of X decreases slowly, eventually stopping at approximately 0.7. This is because gradient descent can only decrease by 1% at a time, making it very slow.

It is worth noting that when $\alpha = 1.01$, the position of X oscillates up and down in a sawtooth pattern and gradually increases. This is because when we input the update function $x_{k+1}=-1.0x_k$, the new x will become -1.02 times the old X . Although it is only a little bit larger each time, as the number of iterations increases, this error will be amplified exponentially.

Q3.1 Gradient Descent on $f(x) = x^2$ with Different Learning Rates



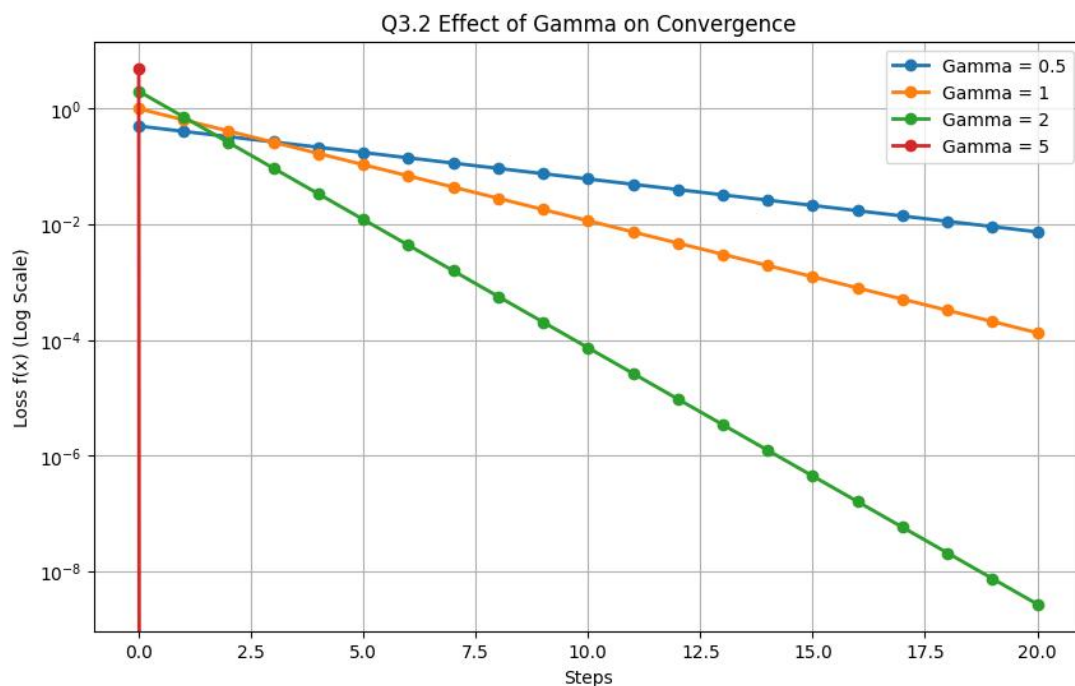
In the corresponding right figure, the loss values of $\alpha = 0.1$ and 0.01 are gradually decreasing, with $\alpha = 0.1$ decreasing at a faster rate and $\alpha = 0.01$ decreasing at a slower rate. Conversely, the loss value of $\alpha = 1.01$ increases, which corresponds to our finding in the left figure that the absolute value of x is gradually increasing.

This demonstrates that the learning rate must be chosen within a reasonable range; an unreasonable learning rate will have the opposite effect.

II. AB

The original function $f(x) = \gamma x^2$, the derivative $f'(x) = 2\gamma x$, and the corresponding gradient descent function $x_{k+1} = x_k - \alpha f'(x_k) = x_k - 0.1(2\gamma x_k)$.

This involves γ controlling the steepness of the function and performing gradient descent on $\gamma \in \{0.5, 1, 2, 5\}$, as shown in the figure below.



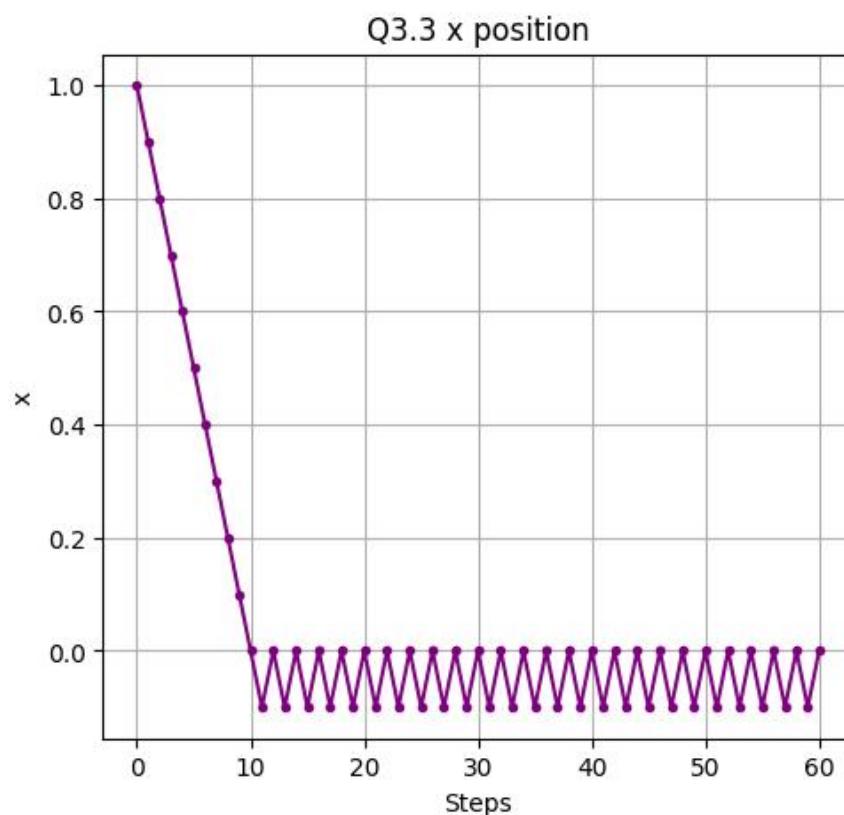
As we can see, when $\text{Gamma} = 0.5, 1, 2$, the larger the value of Gamma , the faster the convergence point. This is because γ it controls the steepness of the function. As γ increases, the terrain becomes steeper and the gradient becomes larger. The same learning rate $\alpha = 0.1$ can produce a larger displacement here, allowing us to get closer to the bottom of the valley more quickly.

However, when $\gamma=5$, the red line is vertical. After the first step, the function value becomes 0 directly. From a mathematical perspective, the updated coefficient is: $1-2\alpha\gamma=1-2(0.1)(5)=1-1=0$, which means that the algorithm jumps directly to the global minimum in one step. This seems to be a coincidence. In this case, the steep terrain actually helps the algorithm because the large gradient just compensates for the small step size, achieving the steepest descent. This indicates that the convergence speed depends on the product of the "step size" and the "steepness of the terrain".

III. AB

$$\text{antiderivative } f(x)=|x|, \text{ derivative } f'(x)=\text{sign}(x)=\begin{cases} 1 & \text{if } x>0 \\ -1 & \text{if } x<0 \\ 0 & \text{if } x=0 \end{cases}$$

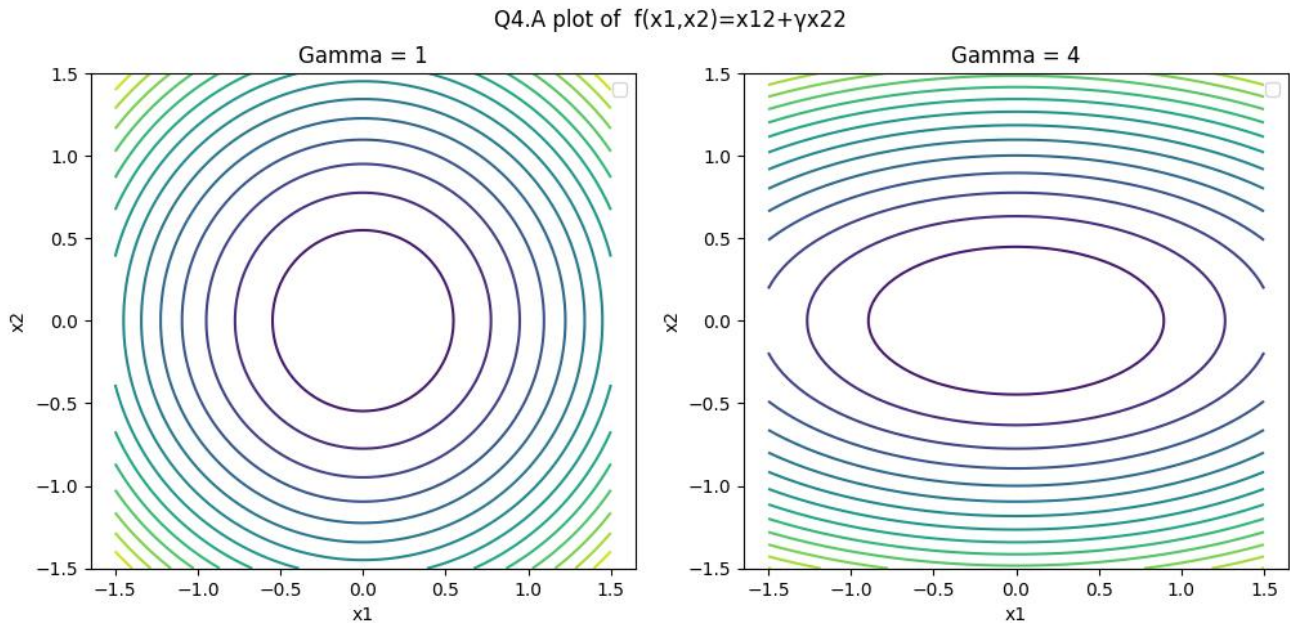
The corresponding gradient descent update formula is : $x_{k+1}=x_k-\alpha \cdot \text{sign}(x_k)$, and the gradient descent result is shown in the figure below.



In the first 10 steps, the value of x decreases at a constant rate because the gradient is always 1 or -1, and the step size is always 0.1, so it steadily decreases by 0.1 in each step. However, when x approaches 0, the graph begins to oscillate, jumping back and forth between 0 and a value slightly less than 0. Mathematically, around the 10th step, the function value drops to 0, and theoretically, the algorithm should converge perfectly and stop at the 10th step. However, at this point, the position of X oscillates. After consulting the literature, I found that this is due to floating-point precision error. 0.1 cannot be precisely represented by a computer, so the 10th step is not equal to 0 in the computer, but rather a very small number. Therefore, the algorithm continues to execute the 11th step $x_{11}=\epsilon-0.1\approx-0.1$ and 12th steps $x_{12}=-0.1-0.1(-1)=-0.1+0.1\approx\epsilon$, and it can be seen that the function gets stuck in a loop, which also explains why the oscillation phenomenon occurs.

A

The antiderivative $f(x_1, x_2) = x_1^2 + \gamma x_2^2$, for $\gamma = 1$ and A contour map was created with $\gamma = 4$, and the result is shown in the figure below.



As can be seen, when Gamma=1, the image appears as a circle because the function at this time corresponds to the standard formula for a circle.

When Gamma=5, the contour lines are elliptical, and the contour lines in the X2 direction are denser, proving that the function is steeper in this direction.

B

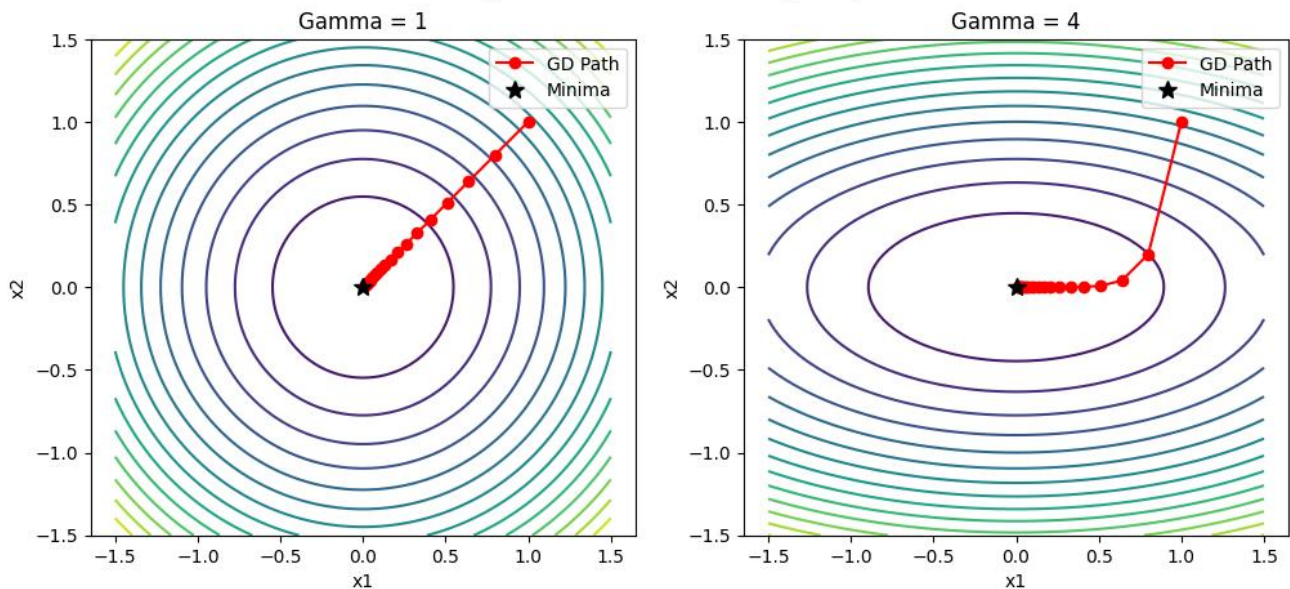
Consistent with Q2, for a function with two parameters, to obtain the gradient descent update function, we first need to take the partial derivatives with respect to the two parameters,

$\frac{\partial f}{\partial x_1} = 2x_1$, $\frac{\partial f}{\partial x_2} = 2\gamma x_2$, so the gradient vector is $\nabla f(x) = \begin{bmatrix} 2x_1 \\ 2\gamma x_2 \end{bmatrix}$, and the update formula

$$\text{is } \begin{bmatrix} x_1^{(k+1)} \\ x_2^{(k+1)} \end{bmatrix} = \begin{bmatrix} x_1^{(k)} \\ x_2^{(k)} \end{bmatrix} - \alpha \begin{bmatrix} 2x_1^{(k)} \\ 2\gamma x_2^{(k)} \end{bmatrix}$$

Based on this, gradient descent was performed for Gamma = 1 and 4 respectively, and the results are shown in the figure below.

Q4.B Effect of Gamma on Trajectory



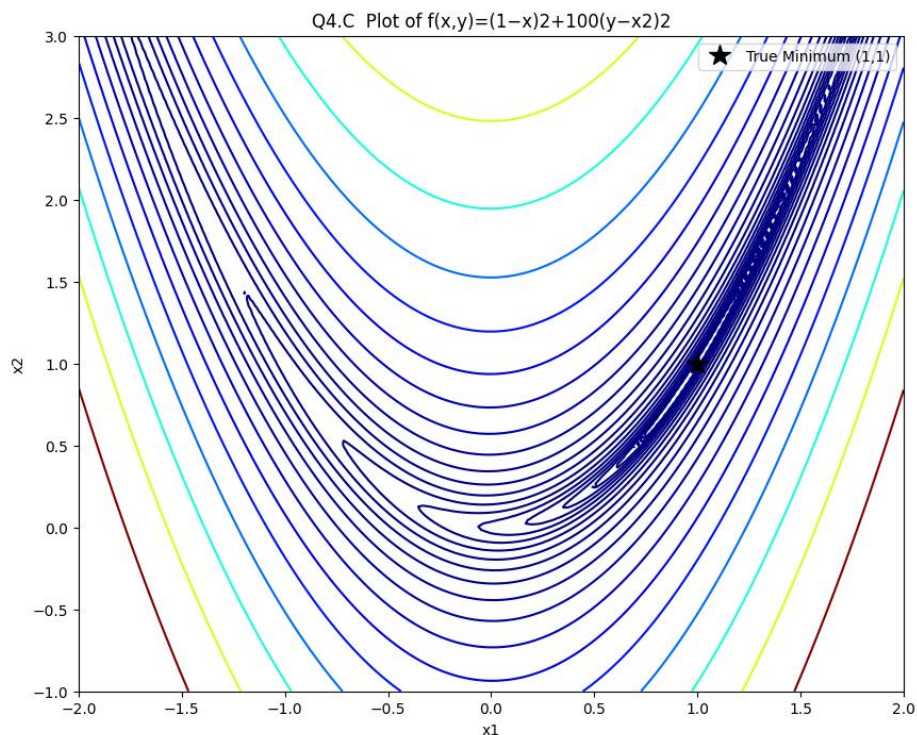
As you can see, when Gamma=1, the red line is a straight line pointing directly to the center of the circle, because this is the perfect condition where the descent speeds in the X1 and X2 directions are the same, and the gradient points directly to the center of the circle.

When Gamma=5, the red line appears as a curve. It can be seen that the convergence speed is very fast in the X2 direction, but slower in the X1 direction. This is because the terrain becomes an ellipse. The X2 direction is 5 times steeper than the X1 direction. The steepness of the function compensates for the short step size, thus accelerating the convergence speed.

This indicates that the actual descent step size is determined by both the 'learning rate' and the 'gradient size'. Steep terrain, by providing a larger gradient, partially compensates for the limitations of a fixed learning rate, thereby accelerating the convergence process in this dimension.

C

The original function $f(x,y)=(1-x)^2+100(y-x^2)^2$ and its contour plot are shown in the figure.



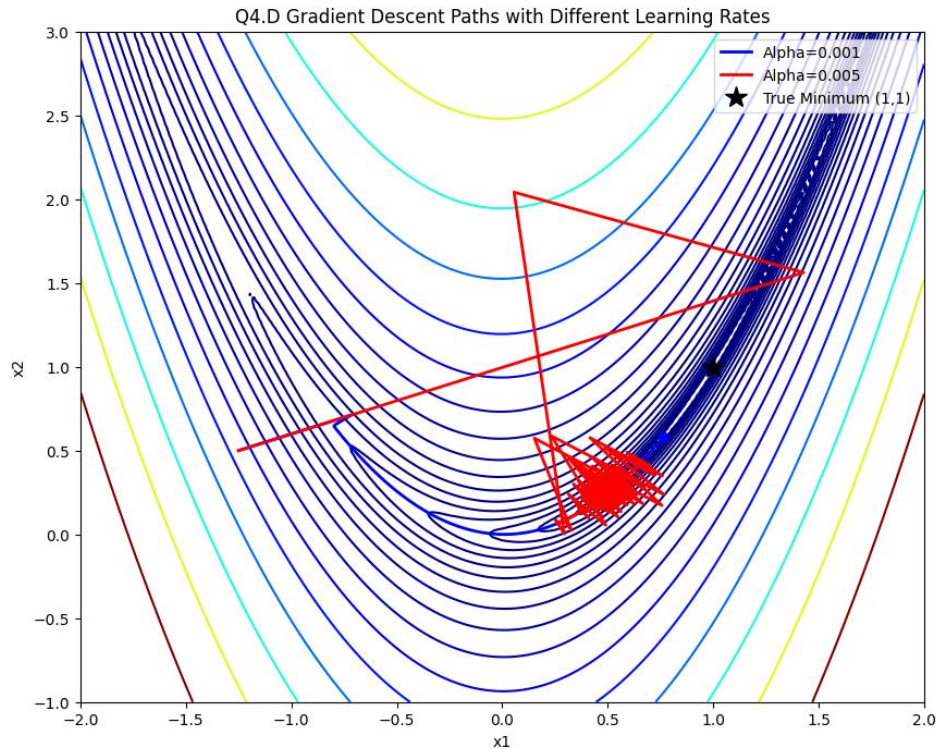
Its ascent path resembles a parabola, and we can determine that the lowest point of the graph is in the range [1,1].

D

First, take the partial derivatives with respect to x and y , respectively $\frac{\partial f}{\partial x} = 2x - 2 - 400xy + 400x^3$,

$\frac{\partial f}{\partial y} = 200(y - x^2)$ to obtain the corresponding gradient vector $\nabla f(x,y) = \begin{bmatrix} -2(1-x) - 400x(y-x^2) \\ 200(y-x^2) \end{bmatrix}$. Then,

perform gradient descent with respect to $\alpha = 0.001$ and $\alpha = 0.005$, respectively. The results are shown in the figure below.



When using a small learning rate $\alpha = 0.001$, the blue line can be seen to form a curve, slowly approaching the lowest point. Despite the limited distance moved in a single step, the algorithm exhibits excellent stability, smoothly sliding down the steep sidewalls into the valley and then slowly advancing towards the lowest point. This small learning rate ensures that each iteration approaches the target through an effective path, and it is believed that with more iterations, it will reach the lowest point.

Conversely, when the learning rate is increased to 0.005, although the step size theoretically increases, the step size quickly leads to violent oscillations perpendicular to the valley direction due to the extremely large gradients of the functions on both sides. The algorithm bounces back and forth between the cliffs on both sides, and a large number of iterations are consumed in ineffective lateral jumps, resulting in the final result not only failing to get closer, but instead remaining at a position further away from the endpoint.

This shows that in functions with strong anisotropy, an excessively large step size will completely sacrifice the longitudinal convergence efficiency by destroying the lateral stability.